SE LAB 4

NAME : BHAVIN G KAMAT
SRN : PES1UG24AM065
SUBJECT : SOFTWARE ENGINEERING

You said:

catch: hook-vs-fish catch detection.

```
def check_catch(hook, fish_list):
    Returns the fish the hook has caught, or
    None.

    for fish in fish_list:
      if abs(hook.y -
        fish.y) < 10:
        return fish
    return None
```

Above code is for catch fish the task is to
A fish is supposed to be caught only when
the hook actually overlaps
>it. In the current build, check_catch (in game/catch. py) only
>compares the hook's depth to the fish's depth, within a fixed
>10-pixel tolerance - it never compares horizontal position at all.
>That means a fish can register as caught while it's still far off to
>one side of the screen, just because the
hook happens to be passing
>through its depth. Fix the check so it's based on the hook and fish

>actually overlapping, not just being at a similar depth.

Give me the appropriate code fix for this function without any irrelvent explation

ChatGPT said:

```
def check_catch(hook, fish_list):
    Returns the fish the hook has
    caught, or None. for fish in
    fish_list:
        abs(hook.x - fish.x) < (hook.width +
        fish.width) / 2
        and abs(hook.y - fish.y) < (hook.height +
        fish.height) / 2
        return
fish return
None
```

You said:

Fish: swims horizontally at a fixed depth, wrapping around when it exits the screen. The starter has one fish type; Task 2 adds more.

```
import pygame


class Fish:
  def finite(self, x, y, speed, width=36, height=18, point_value=10, color=(80, 180, 220)):
    self.x = float(x)
    self.y = y
    self.speed = speed
    self.width = width
    self.height = height
    self.point_value = point_value
    self.color = color

  def update(self, screen_width): self.x += self.speed
```

```
        if self.speed > 0 and self.x >
          screen_width: self.x = -
          self.width
        elif self.speed < 0 and self.x < -
          self.width: self.x =
          screen_width

    def
      get_rect(sell:
      return
      pygame.Rect(
        int(self.x - self.width / 2), int(self.y -
        self.height / 2), self.width,
        self.height,
```

Above code is for fish. The task is to

Task 2: Implement multiple fish types

>Introduce at least two fish types that differ in movement speed and

>point value (for example, a slow low-value fish and a fast

>high-value one). Give each type its own look (color and/or size) So

>they're visually distinguishable, and make sure the correct point

>value is awarded when each type is caught.

Give me the appropriate code fix for this function without any

irrelvent explation\

ChatGPT said:

```
import pygame


class Fish:
   def finite(
      self, x, y, speed,
      width=36, height=18,
      point_value=10, color=(80,
      180, 220)

      self.x =
      float(x)
      self.y = y
      self.speed =
      speed
      self.width =
      width
      self.height
      = height
      self.point_value =
```

```
      point_value self.color
      = color

   def update(self,
      screen_width):
      self.x +=
      self.speed

      if self.speed > 0 and
         self.x › screen_width:
         self.x = -self.width
      elif self.speed < 0 and
         self.x < -self.width:
         self.x = screen_width

   def
      get_rect(sel
      f): return
      pygame.Rect(
         int(self.x -
         self.width / 2),
         int(self.y -
         self.height / 2),
         self.width,
         self.height,



class SlowFish(Fish):
   def finite(self, x, y,
      speed=1, **kwargs):
```

```
super().finite(
   x, y, speed,
```

```
        width=36,
        height=18
        ,
        point_val
        ue=10,
        color=(80, 180, 220),



class FastFish(Fish):
  def finite(self, x, y, speed=4, **kwargs):
    super().fi
      nite( x,
      y,
      speed,
      width=48
      ,
      height=2
      4,
      point_val
      ue=30,
      color=(255, 100, 80),
```

You said:

GameEngine: owns the hook and the fish, and runs one frame's worth of

game logic.

Starter version: the hook casts and retracts automatically in a continuous loop - there's no player control over casting yet (that's Task 3), only one fish type exists (Task 2 adds more), and there's no round timer (Task 4). Catch detection also has a known bug (see game/catch.py) that Task 1 asks you to fix.

```
from game.hook import
Hook, IDLE from
game.fish import Fish
from game.catch import check_catch
from game.renderer  import WIDTH,
HEIGHT, SURFACE_Y, MAX  DEPTH  Y


class
  GameEngi
  ne: def
  finite(sew:
    self.hook = Hook(x=WIDTH / 2,
surface_y=SURFACE_Y,
max_depth_y=MAX_DEPTH_Y, speed=5)
```

```
        self.fish_list = [
            Fish(x=100, y=180, speed=2,
    point_value=10, color=(80, 180, 220)),
            Fish(x=400, y=280, speed=-2,
    point_value=10, color=(80, 180, 220)),
            Fish(x=250, y=380, speed=3,
    point_value=10, color=(80, 180, 220)),

        self.hooked_fish =
        None self.score  =
        0

    def update(sell:
        if self.hook.state
            == IDLE:
            self.hook.start
            _cast()

        self.hook.update()

        for fish in
            self.fish_list:
            fish.update(WID
            TH)

        if self.hooked_fish is
            not None:
            self.hooked_fish.x =
            self.hook.x
            self.hooked_fish.y =
```

```
            self.hook.y if
            self.hook.state ==
            IDLE:
                self.score +=
                self.hooked_fish.point_value
                self.hooked_fish = None
        else:
            caught =
            check_catch(self.hook,
            self.fish_list) if caught is not
            None:
                self.fish_list.remove(caugh
                t) self.hooked_fish =
                caught self.hooked_fish.x
                = self.hook.x
                self.hooked_fish.y =
                self.hook.y
                self.hook.catch_fish()

    def draw(self, surface,
        font): from game
        import renderer
        draw_list =
        list(self.fish_list)
        if self.hooked_fish is not None:
            draw_list.append(self.hooked_fish)
        renderer.draw_scene(surface, self.hook,
        draw_list)
```

renderer.draw_text(surface, font, f"Score: (self.score)", (10, 10))

Above code is for game_engine.

The task is to ### **Task 3:**

Implement player-controlled casting

> Change the hook so the player decides when it casts, instead of it
> looping automatically. Pressing a key should start a cast if the
> hook is currently idle; it should then travel down and return on its
> own (already handled) once it reaches maximum depth or catches a
> fish. A new cast should not be able to interrupt one that's already
> in progress.

Task 4: Implement a 30-second round timer

> Add a 30-second countdown for the round. Display the remaining time
> on screen. Once it reaches zero, no further catches should be
> possible, the round should end, and the final score should be shown
> clearly. Provide a way to start a new round with the score and timer
> both reset.

ChatGPT said:

```
import pygame

from game.hook import Hook, IDLE
from game.fish import Fish,
SlowFish, FastFish from
game.catch import
check_catch
from game.renderer import WIDTH, HEIGHT,
SURFACE_Y, MAX_DEPTH_Y


class GameEngine:
   ROUND_DURATION =
   30

   def
      finite(self)
      : self.hook
      = Hook(
         x=NIDTH / 2,
         surfacep=SURFACE_Y
         ,
         max_depth_y=MAX_DE
         PTH_Y, speed=5,
```

```
        self.score = 0
        self.remaining_time =
        self.ROUND_DURATION
        self.round_over = False
        self.start_time =
        pygame.time.get_ticks()
        self.hooked_fish = None
        self.reset_fish()

d       def
e
f









d
e
f


        def
```

```
rese    , speed=1), FastFish(x=400,
t_fi    y=280, speed=-4),
sh(s    SlowFish(x=250, y=380,
elf)    speed=1), FastFish(x=550,
:       y=220, speed=-3),
self
.fis
h_li
st = handle_event(self, event):
[       if event.type == pygame.KEYDOWN:
  S       if event.key == pygame.K_SPACE:
  l         if not self.round_over and self.hook.state
  o           == IDLE: self.hook.start_cast()
  w
  F       elif event.key ==
  i         pygame.K_r: if
  s         self.round_over:
  h           self.new_round()
  (
  x   new_round(self):
  =   self.hook =
  1       Hook(
  0       x=WIDTH / 2,
  0       surfacep=SURFACE_Y,
  ,       max_depth_y=MAX_DEPT
          H_Y, speed=5,
  y
  =   self.score = 0
  1   self.remaining_time      =
  8   self.ROUND_DURATION
  0   self.start_time          =
      pygame.time.get_ticks()
```

```
s    oked_fish    =
e    None
l    self.reset_fis
f    h()
.
r    update(self):
o    if
u       self.round_ov
n       er: return
d
_    elapsed = (pygame.time.get_ticks() -
o    self.start_time) / 1000 self.remaining_time =
v    max(0, self.ROUND_DURATION - elapsed)
e
r


=


F
a
l
s
e
s
e
l
f
.
h
o
```

```
if
   self.remaining_ti
   me <= 0:
   self.round_over =
   True
   self.hooked_fish
   = None return

self.hook.update()

for fish in self.fish_list:
   fish.update(WIDTH)

if self.hooked_fish is not
   None: self.hooked_fish.x
   = self.hook.x
   self.hooked_fish.y =
   self.hook.y

   if self.hook.state == IDLE:
      self.score +=
      self.hooked_fish.point_value
      self.hooked_fish = None
else:
   caught = check_catch(self.hook,
   self.fish_list)

   if caught is not None:
      self.fish_list.remove(
      caught)
```

```
            self.hooked_fish =
            caught
            self.hooked_fish.x =
            self.hook.x
            self.hooked_fish.y =
            self.hook.y
            self.hook.catch_fish()

def draw(self, surface, font):
   from game import renderer
   draw_list =
   list(self.fish_list)
   if self.hooked_fish
   is not None:
      draw_list.append(self.hooked_fish)

   renderer.draw_scene(surface, self.hook,
   draw_list) renderer.draw_text(
      surface, font, f”Score: {self.score}“, (10,
      10)

   renderer.dra
      w_text(
      surface,
      font,
      f“Time:
      {int(self.remaining_time)
      }s“, (10, 40),
```

```
    if self.round_over:
        renderer.draw_text(
            surface
            , font,
            f”ROUND OVER! Final Score:
            {self.score}“, (WIDTH // 2
            - 150, HEIGHT // 2 - 20),

        renderer.draw_text(
            surf
            ace,
            font
            ,
            "Press R to start a
            new round", (WIDTH //
            2 - 150, HEIGHT // 2 +
            20),
```

In your main game loop, pass keyboard events to the engine:

```
for event in
    pygame.event.get()
    : if event.type ==
    pygame.QUIT:
        running =
    False else:
```

```
    engine.handle_event(event)
```

Press Space to cast and R to restart after the round ends.

You said:

Hook: the player's fishing hook. Moves only vertically, at a fixed horizontal position. In the starter, it casts and retracts on its own, in a continuous loop, with no player input at all - Task 3 changes this to be player-controlled.

```
import pygame

IDLE = "idle"
CASTING =
"casting"
RETRACTING =
"retracting"


class Hook:
  def finite(self, x, surface_y, max_depth_y,
  speed=4, width=14,
```

```
height=14):
    self.x = x
    self.surface_y =
    surface_y
    self.max_depth_y =
    max_depth_y self.y =
    surface_y
    self.speed =
    speed self.width
    = width
    self.height =
    height self.state =
    IDLE

  def
    start_cast(sell:
    self.state =
    CASTING self.y
    = self.surface_y

  def update(sell:
    if self.state ==
      CASTING:
      self.y +=
      self.speed
      if self.y >=
        self.max_depth_y:
        self.y =
        self.max_depth_y
        self.state =
        RETRACTING
    elif self.state ==
```

```
            RETRACTING:
            self.y -= self.speed
            if self.y <=
              self.surface_y:
              self.y =
              self.surface_y
              self.state = IDLE

        def catch_fish(sell:
          """Called when a fish has been caught -
      immediately head back up."""
          self.state = RETRACTING

        def
          get_rect(sell
          : return
          pygame.Rect(
            int(self.x - self.width / 2), int(self.y -
            self.height / 2), self.width, self.height,
```

Identfy the bug in this file and give me the fix

ChatGPT said:

The bug is that **start_cast() allows a** new cast even when the hook is already **CASTING** or `RETRACTING.`

Fix:

```
def start_cast(self):
   if self.state
      != IDLE:
      return

   self.state =
   CASTING self.y
   =
   self.surfaces
```

This ensures a new cast starts only when the hook is idle, preventing an ongoing cast from
being interrupted.

You said:

renderer: all pygame drawing lives here, kept separate from game logic.

```
import pygame

WIDTH, HEIGHT = 700, 500
SURFACE_Y = 80
MAX_DEPTH_Y =
HEIGHT - 40

WINDOW_SIZE =

(WIDTH, HEIGHT)
```

```
COLOR_SKY = (140,
200, 230)
COLOR_WATER = (30, 90, 150)
COLOR_BOAT = (120, 80, 50)
COLOR LINE = (240, 240, 240)
COLOR_HOOK = (220, 220, 220)
COLOR TEXT = (255, 255, 255)


def draw_scene(surface, hook, fish_list):
  surface.fill(COLOR_SKY, pygame.Rect(0, 0,
  WIDTH, SURFACE_Y)) surface.fill(COLOR_WATER,
  pygame.Rect(0, SURFACE_Y, WIDTH,
HEIGHT - SURFACE_Y))

  pygame.draw.rect(surface,
COLOR_BOAT, (hook.x - 40,
SURFACE_Y - 20, 80, 22))

  pygame.draw.line(surface, COLOR_LINE, (hook.x,
SURFACE_Y), (hook.x, hook.y), 2)
```

```
    pygame.draw.circle(surface,
COLOR_HOOK, (int(hook.x), int(hook.y)), 7)

    for fish in fish_list:
     pygame.draw.ellipse(surface, fish.color,
                         fish.get_rect())


def draw_text(surface, font, text, pos,
    color=COLOR_TEXT):
    surface.blit(font.render(text, True, color),
    pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect =
surf.get_rect(center=(surface.get_width() yf 2,
surface.get_height() // 2))
    surface.blit(surf, rect)
```

Identify the bug here and give me the approriate fix

ChatGPT said:

The bug is that `draw_scene()` does not display a clear round-over banner, and `draw_banner()` is defined but never used. Also, the hook's floating-point `x` and `y` coordinates should be converted to integers for Pygame drawing.

Fix:

```
def draw_scene(surface, hook,
   fish_list): surface.fill(COLOR_SKY,
   pygame.Rect(0, 0, WIDTH, SURFACE_Y))
   surface.fill(
      COLOR_WATER,
      pygame.Rect(0, SURFACE_Y, WIDTH, HEIGHT -
      SURFACE_Y),


   pygame.draw.
      rect(
      surface,
      COLOR_BOA
      T,
      (int(hook.x - 40), SURFACE_Y - 20, 80, 22),


   pygame.draw.
      line(
      surface,
      COLOR_LIN
      E,
      (int(hook.x),
      SURFACE_Y),
      (int(hook.x),
      int(hook.y)),
```

```
      2,


   pygame.draw.
      circle(
      surface,
      COLOR_HOO
      K,
      (int(hook.x),
      int(hook.y)), 7,


   for fish in fish_list:
      pygame.draw.ellipse(surface, fish.color,
      fish.get_rect())


def draw_text(surface, font, text, pos,
   color=COLOR_TEXT):
   surface.blit(font.render(text, True, color),
   pos)


def draw_banner(surface, font, text):
   surf = font.render(text,
   True, (255, 220, 80)) rect =
   surf.get_rect(
      center=(surface.get_width() // 2,

   surface.get_height() // 2) surface.blit(surf,
```

```
rect)
```

To display the round-over banner, call drax_banner(sur-Face, I-ont, "ROUND OVER!") from your game engine's draw() method when self. round_over is True .

You said:

```
Fishing Game (Lab Starter)

Run with: python3 main.py


import pygame

from game.game_engine
import GameEngine from
game.renderer import
WINDOW_SIZE


def main():
  pygame.in
  it()
```

```
screen =
pygame.display.set_mode(WINDO
W_SIZE)
pygame.display.set_caption("Fis
hing")
clock = pygame.time.Clock()
font =
pygame.font.SysFont("Conso
las", 22) for event in
pygame.event.get():
  if event.type ==
    pygame.QUIT:
    running = False
  else:
    engine.handle_event(event)

engine =
GameEngine()
running = True
while running:
  for event in
    pygame.event.get(
    ): if event.type ==
    pygame.QUIT:
      running = False

  engine.update()
  engine.draw(scree
  n, font)

  pygame.display.flips
```

clock.tick(60)

pygame.quite

if enamel == "
main '.
main()

Identify the bug here and give me the appripriate fixes

ChatGPT said:

The bug is that eng1ne and runn1ng are used before initialization, and the main game loop

does not pass keyboard events to engine. hand1e_event(event) .

Fixed main.py :

Fishing Game (Lab

Starter) Run with:

python3 main.py

```
import pygame

from game.game_engine
import GameEngine from
game.renderer import
WINDOW_SIZE


def main():
   pygame.init()
   screen = pygame.display.set_mode(WINDOW_SIZE)
   pygame.display.set_caption(”Fishing”)
   clock = pygame.time.Clock()
   font = pygame.font.SysFont(”consolas“, 22)

   engine =
   GameEngine()
   running = True

   while running:
      for event in
         pygame.event.get(
         ): if event.type
         == pygame.QUIT:
            running =
         False else:
            engine.handle_event(event)

      if not
         runnin
         g:
```

```
            break

        engine.update()
        engine.draw(screen, font)

        pygame.display.f
        lip()
        clock.tick(60)

    pygame.quit()


if enamel == ”
    main ”: main()
```

You said:

Export these above conversion as a pdf

Analyzed